

1. RTL Design (alu.v)

```
module alu (  
    input [7:0] A,  
    input [7:0] B,  
    input [2:0] ALU_Sel,  
    output reg [7:0] Result,  
    output Zero  
);  
  
always @(*) begin  
    case(ALU_Sel)  
  
        3'b000: Result = A + B;    // Addition  
  
        3'b001: Result = A - B;    // Subtraction  
  
        3'b010: Result = A & B;    // AND  
  
        3'b011: Result = A | B;    // OR  
  
        3'b100: Result = A ^ B;    // XOR  
  
        3'b101: Result = ~A;       // NOT  
  
        3'b110: Result = A << 1;   // Left Shift  
  
        3'b111: Result = A >> 1;   // Right Shift  
  
        default: Result = 8'd0;  
  
    endcase  
end  
  
assign Zero = (Result == 8'd0);  
  
endmodule
```

2. Testbench (alu_tb.v)

```
`timescale 1ns/1ps
```

```
module alu_tb;
```

```
reg [7:0] A;
```

```
reg [7:0] B;
```

```
reg [2:0] ALU_Sel;
```

```
wire [7:0] Result;
```

```
wire Zero;
```

```
integer i;
```

```
alu DUT (
```

```
    .A(A),
```

```
    .B(B),
```

```
    .ALU_Sel(ALU_Sel),
```

```
    .Result(Result),
```

```
    .Zero(Zero)
```

```
);
```

```
initial begin
```

```
    $dumpfile("alu.vcd");
```

```
    $dumpvars(0, alu_tb);
```

```
    //-----
```

```
    // Directed Test Cases
```

```
    //-----
```

```
A = 10; B = 5; ALU_Sel = 3'b000;  
#10;
```

```
A = 10; B = 5; ALU_Sel = 3'b001;  
#10;
```

```
A = 8'hAA; B = 8'h55; ALU_Sel = 3'b010;  
#10;
```

```
A = 8'hAA; B = 8'h55; ALU_Sel = 3'b011;  
#10;
```

```
A = 8'hAA; B = 8'h55; ALU_Sel = 3'b100;  
#10;
```

```
A = 8'h0F; B = 0; ALU_Sel = 3'b101;  
#10;
```

```
A = 8'h08; B = 0; ALU_Sel = 3'b110;  
#10;
```

```
A = 8'h80; B = 0; ALU_Sel = 3'b111;  
#10;
```

```
//-----  
// Random Test Cases  
//-----
```

```
for(i=0;i<100;i=i+1)  
begin  
    A = $random;  
    B = $random;  
    ALU_Sel = $random % 8;  
    #10;  
end
```

\$finish;

end

endmodule

Coverage achieved:

- **Directed operation coverage:** 100% (all 8 ALU operations exercised)
- **Random testing:** 100 randomized input combinations across all operations
- **Corner cases covered:** Zero result, alternating bit patterns (0xAA, 0x55), shift operations, and arithmetic operations

3. Simulation

Using ModelSim

Compile :-

vlog alu.v

vlog alu_tb.v

Run :-

vsim alu_tb

run -all

View waveform :-

add wave *

Using Icarus Verilog + GTKWave

Compile :-

iverilog -o alu alu.v alu_tb.v

Run :-

vvp alu

Open waveform :-

gtkwave alu.vcd

Coverage achieved:

- **Directed operation coverage:** 100% (all 8 ALU operations exercised)
- **Random testing:** 100 randomized input combinations across all operations
- **Corner cases covered:** Zero result, alternating bit patterns (0xAA, 0x55), shift operations, and arithmetic operations

waveform

